

C A R I A D

RTM Cookbook

Version: 1.0.0

Revision History:

Author	Date	Version	Remark
Sai Kishan M H	23-07-2024	1.0.0	Initial Version

Contents

1. Purpose and Scope	3
2. Introduction	4
3. Add module in Basic Editor	5
4. Configuration of Runtime Measurement Section	7
4.1. GetCurrentTime	13
5. Developer Steps	14
6. Task Mapping & Module Initialization	18
7. Coding Part	21
8. Makefile changes for vector build environment.	23
9. Compilation	24
10. Flashing ECU using TRACE32 debugger	25
11. Issues observed and their solutions	27
12. Contact	28

Runtime Measurement Integration Steps

Integrated RTM Software

Reference Documents:

[1] TechnicalReference_Rtm.pdf

[2] UserManual_AMD.pdf

Third Party Vendor Name: Vector

1. Purpose and Scope

The purpose of this document is to provide an integrator with a technical manual for integrating the AUTOSAR BSW module RTM. This document outlines the integration steps for RTM.

Integration Requirements:

Following prerequisites must be available for the integration of the RTM implementation:

[1]. Used Tool Chains :

Vector DaVinci Configurator (BSW) Version: 5.26.35 (SP2)

Vector DaVinci Developer Classic Version: 4.11 (SP1)

[2]. Build environment for including source code of RTM

[3]. AUTOSAR toolchain for configuring BSW and RTE

Follow the steps given in subsequent chapters to Integrate RTM module.

2. Introduction

AMD – AUTOSAR Monitoring and Debugging:

- The AMD (AUTOSAR Monitoring and Debugging) includes the modules vDbg (DeBuGging) and Dlt (Diagnostic Log and Trace) as specified by AUTOSAR, and it is further enhanced by Vector with the addition of the vRtm (RunTime Measurement) module.
- Currently, we are integrating AMD Runtime Measurement (vRtm).
- With AMD Runtime Measurement (vRtm), you can measure the runtime and CPU load of:
 - Predefined functions of the BSW modules
 - Your application code
- Runtime Measurement (RTM) module allows the user to determine runtimes and CPU load of BSW modules and user code sections.
- Measurement is controlled- and evaluated in CANoe by RTM's frontend or a self-written RTM application.
- RTM provides a set of macros, which are used to instrument in the source code to be measured. Such an instrumented code section is called measurement point (MP).

How Does Measurement Work:

- At a start point the measurement is started at a stop point, it is stopped. In code the integrator can define as many start-stop couples as you want. You can also use the predefined measurements of the BSW modules.
- Define the complete duration of a measurement and get the following results:
 - *Count*: Counts how often the defined start-stop sections had been passed.
 - *Max*: Shows the absolute longest runtime of each start-stop section.
 - *Min*: Shows the absolute shortest runtime of each start-stop section.
 - *Average*: Shows the average runtime of each start-stop section.
 - *Average CPU load*: Shows the average CPU load caused by each start-stop section.

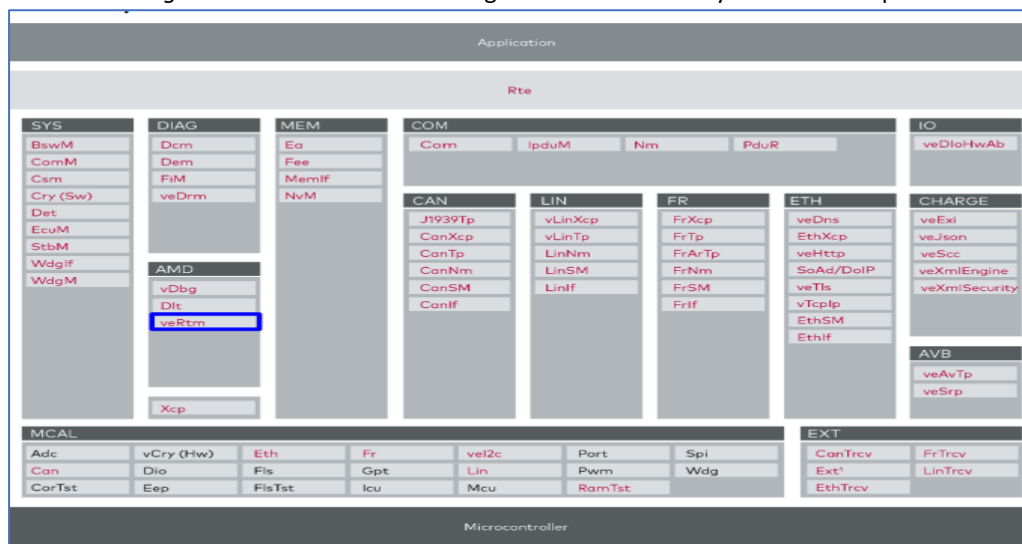


Figure 1

3. Add module in Basic Editor

In Vector DaVinci Configurator go to below mentioned path:

- i. Project -> Project Settings -> Modules -> Click ADD “+” button.

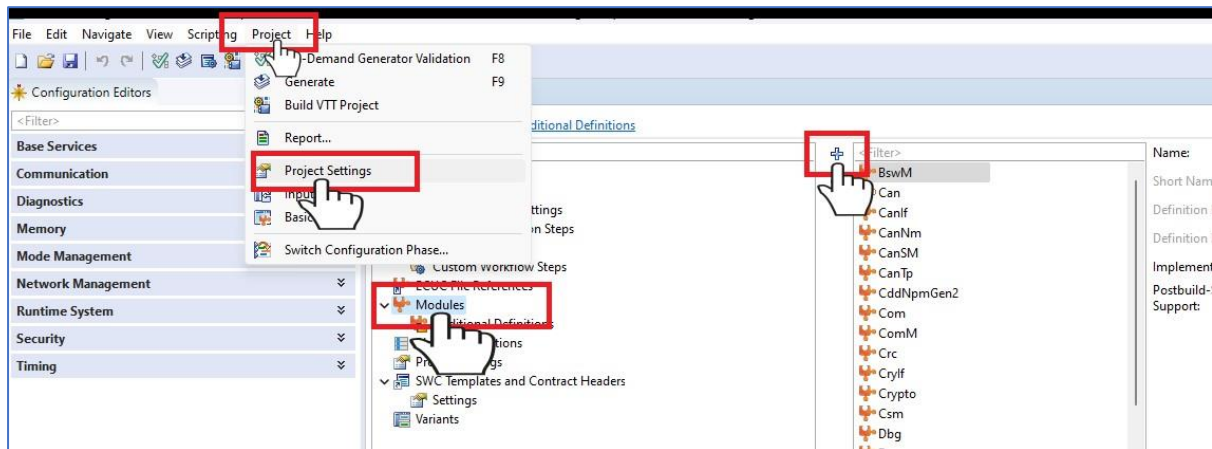


Figure 2

- ii. Select option “Software Integration Package (SIP)” and click next.

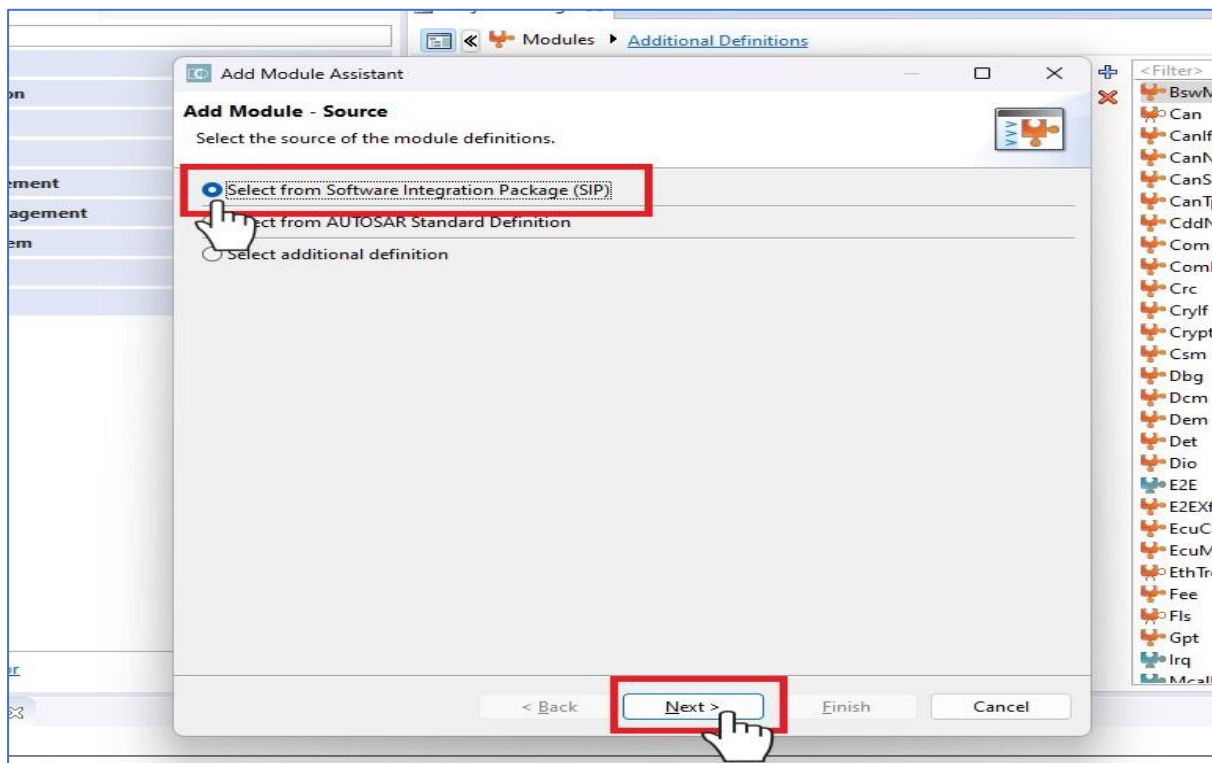


Figure 3

- iii. Search and select Rtm module then Click NEXT -> Finish.

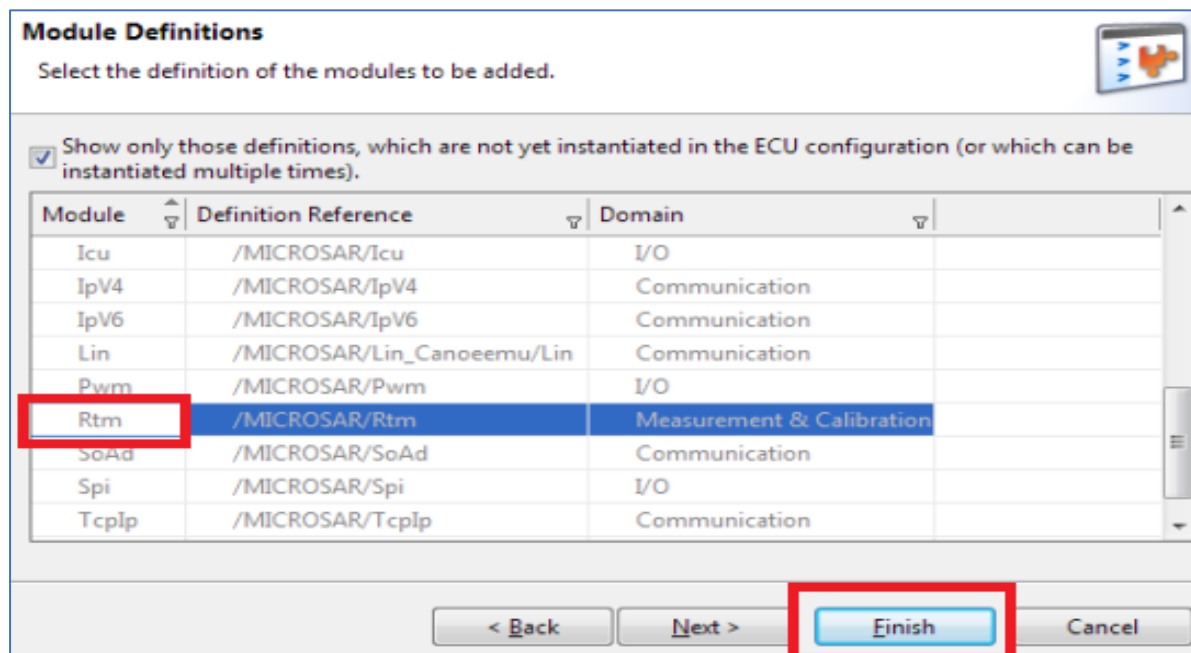


Figure 4

Now you should see the vRtm module in the list of modules.

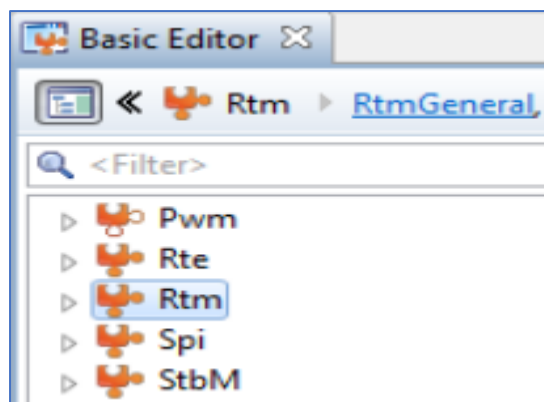


Figure 5

General Information about CPU Load Measurement:

- RTM provides functionality to measure the current overall CPU load across all cores. There are two methods to control CPU load measurement, selectable at pre-compile time using the DaVinci Configurator parameter: `/MICROSAR/Rtm/RtmCpuLoadMeasurement/RtmCpuLoadControlMode``.
- The available CPU load control modes are:
 - C_API
 - Xcp
- In this context, we are using the C_API method for calculating the CPU load measurement.
- For more information about CPU Load Measurement refer chapter 2.1.11.3.1.

4. Configuration of Runtime Measurement Section

i. For CPU Load Measurement calculation,

- Go to the Basic Editor and open the Rtm tree.
- For Calculating the CPU Load, the integrator must create the subcontainer for RtmCpuLoadMeasurement by following the below given path,
/MICROSAR/Rtm -> 'right click' on Rtm container -> Create RtmCpuLoadMeasurement Container.
After creating the subcontainer, configure the parameters as shown in the image below.

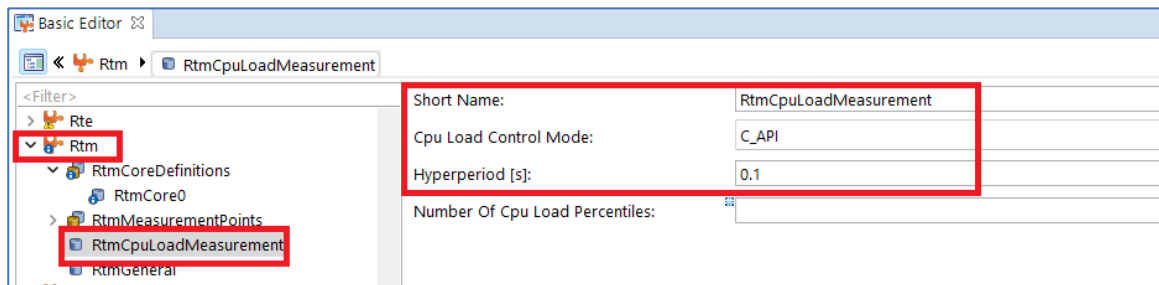


Figure 6

- After successfully adding the submodule, several general errors may arise. The integrator has to create new 'RtmMeasurementPoints' for 'RtmCpuLoadMeasurement'. To resolve the errors, follow the remaining steps outlined below.

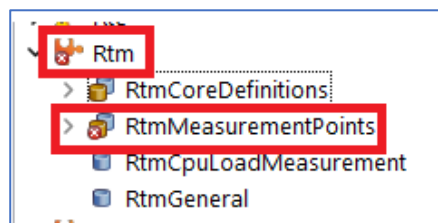


Figure 7

- Select RtmMeasurementPoints and add a new measurement point via a click on the blue plus.

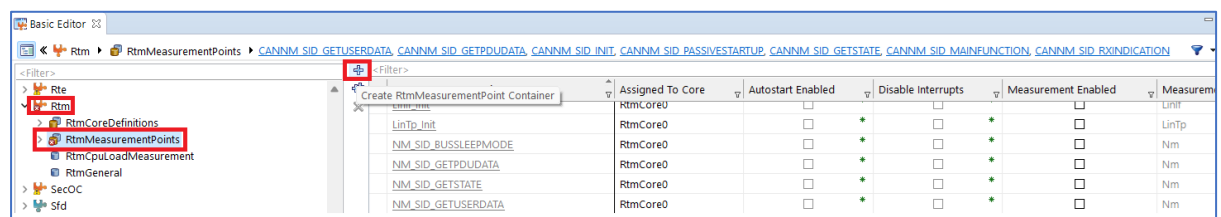


Figure 8

- Based upon the 'RtmCoreDefinitions', a separate MP is added for 'Rtm_CpuLoadMeasurement'. Select the new RtmMeasurementPoint and enter the following values.
RtmMeasurementPointType → CPU_Load.
RtmMeasurementType → GrossExecutionTime and assign the measurement point to 'RtmCore0'.

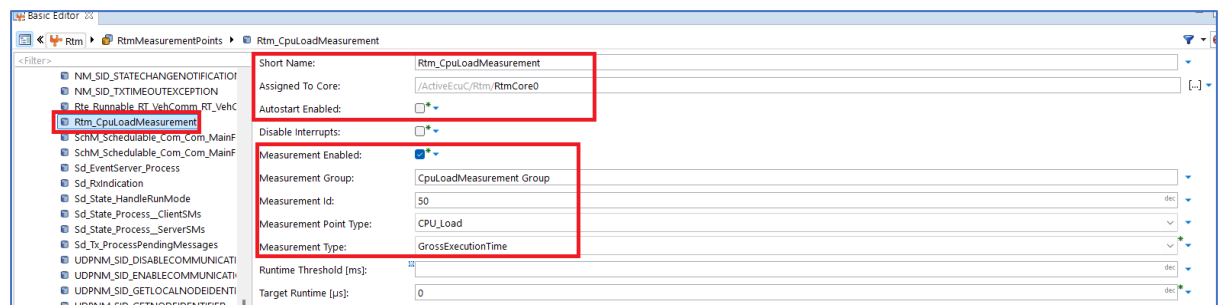


Figure 9

The integrator has to enable the measurement point to get generated in Rtm_Cfg.h, by following the path: */MICROSAR/Rtm/RtmMeasurementPoint/RtmMeasurementEnabled*.

- For CpuLoadMeasurement, the container '*/MICROSAR/Rtm/RtmCpuLoadMeasurement*' exists and the 'Rtm.h' is added to the OS configuration. This feature uses the OS timing hook 'OS_VTH_SCHEDULE' which is automatically generated in the 'Rtm_Cfg.h'. It is implemented to start the CPU load MP when the idle/background task is terminated, preempted, or it must wait for an event. The CPU load MP is stopped when the idle/background task is activated, resumed, or the event is set.

Configuration of Os Container:

- For configuring OS follow the below mentioned path, *MICROSAR/Os/OsOS* -> right click on the container and create Sub container -> then select 'OsDebug' container.

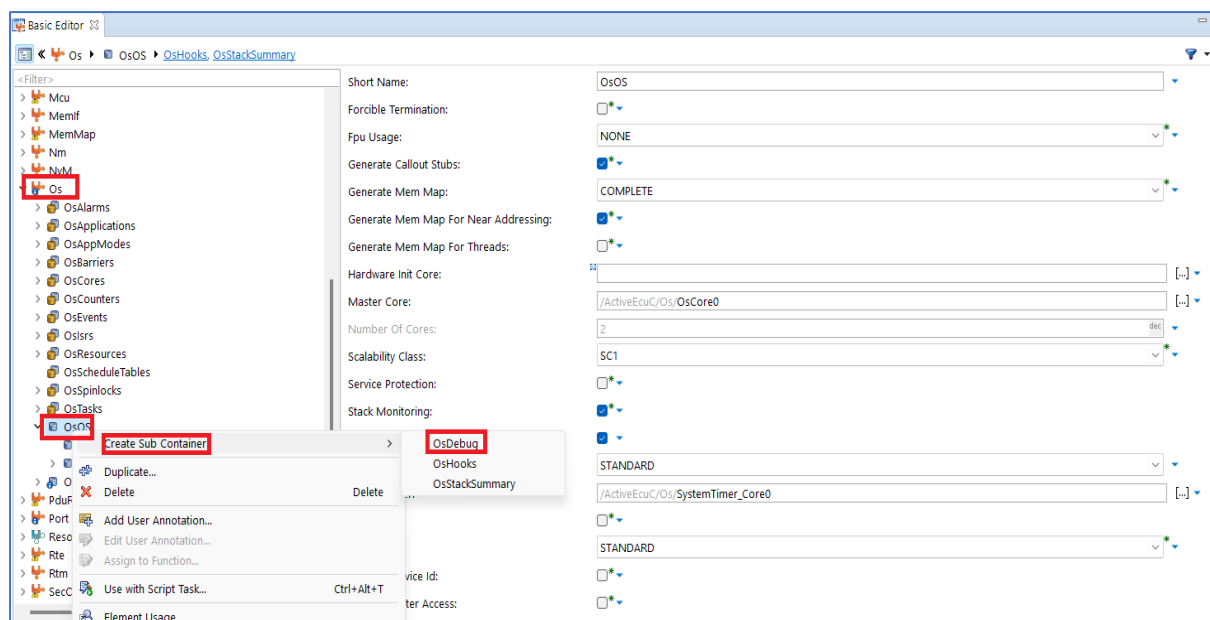


Figure 10

- After creating the subcontainer, integrator has to configure the 'OsDebug' container by using the below image.

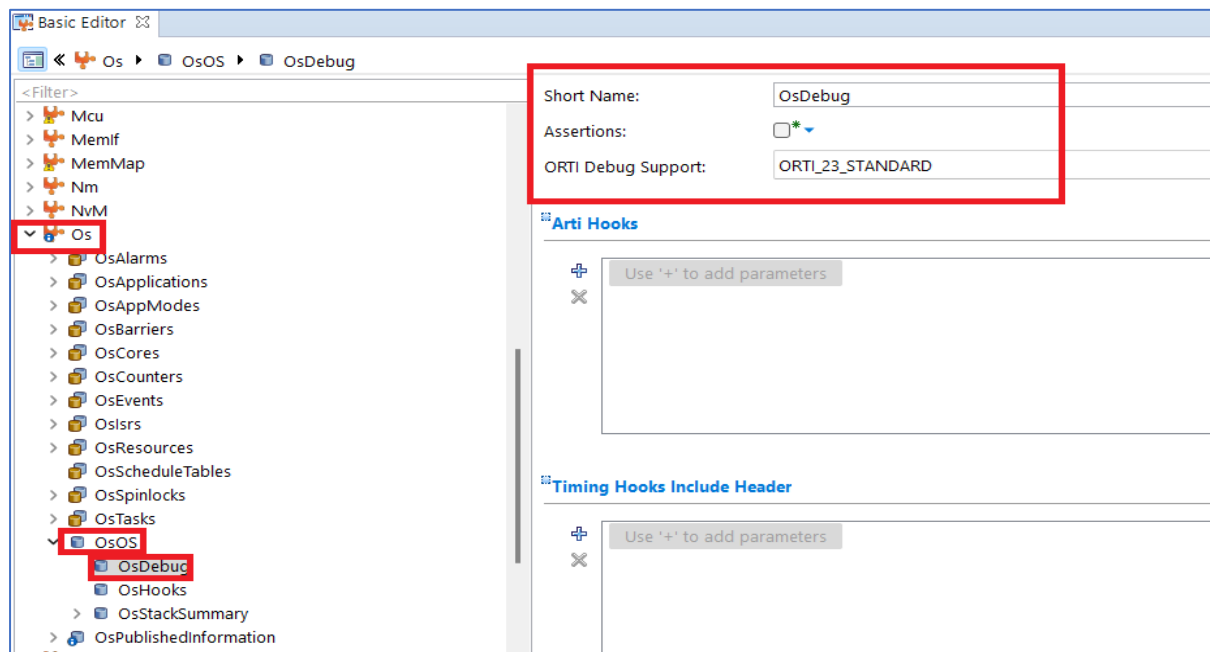


Figure 11

- The integrator has to include the '<Rtm.h>' file in 'Timing Hooks Include Header' parameter via a click on the blue plus.

By using below path configure the Timing Hooks Include Header:

/MICROSAR/Os/OsOS/OsDebug/OsTimingHooksIncludeHeader.

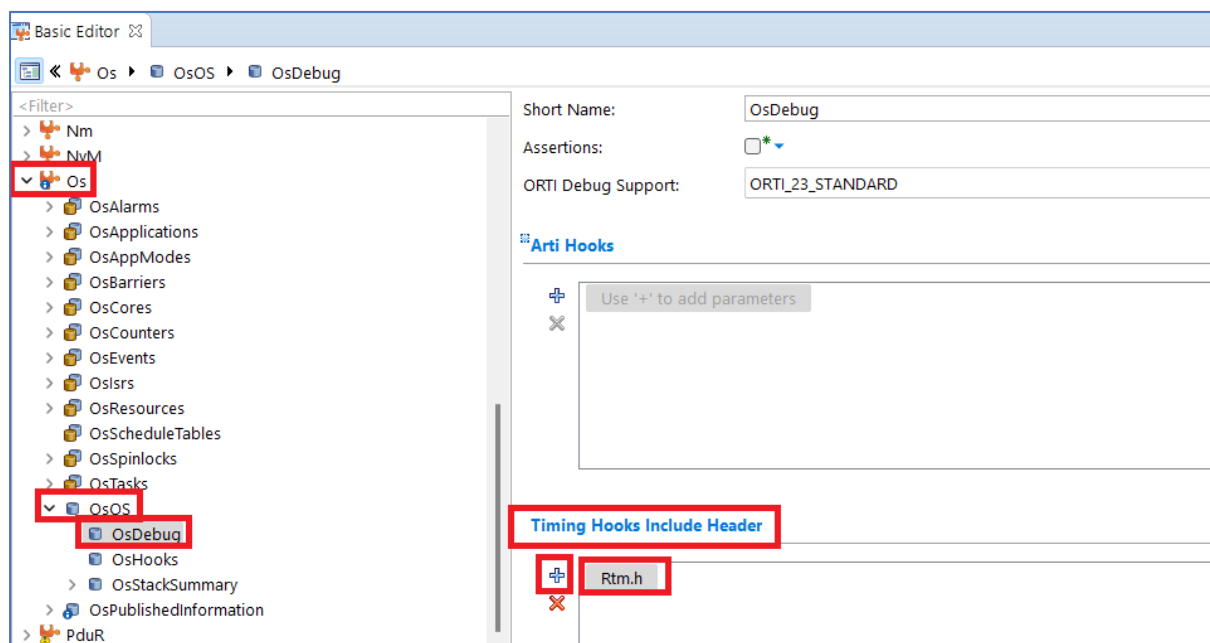


Figure 12

RtmCoreDefinitions Container:

1. For RTM, configure the RtmCoreDefinition container by following the given path:
/MICROSAR/Rtm/RtmCoreDefinition/RtmCore0
When configuring the RtmCoreDefinition container for RTM, the integrator should select 'BswBackgroundTask' as the 'Background Task Name' if any functions or runnables are associated with it. Otherwise, we should use 'IdleTask_OsCore<CoreId>' as the 'Background Task Name'.
2. For 'Background Task Name', the custom background/idle task shall be set here.

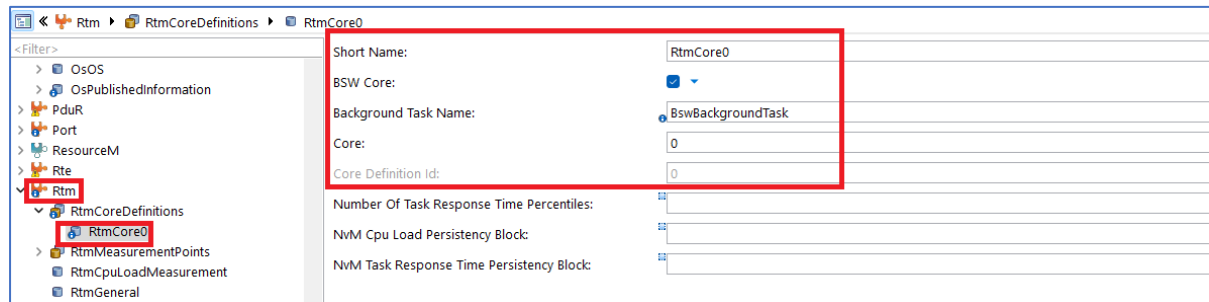


Figure 13

RTM General Container:

1. This container contains the General configuration parameters of the RTM. Do the following configurations as shown in the image below.

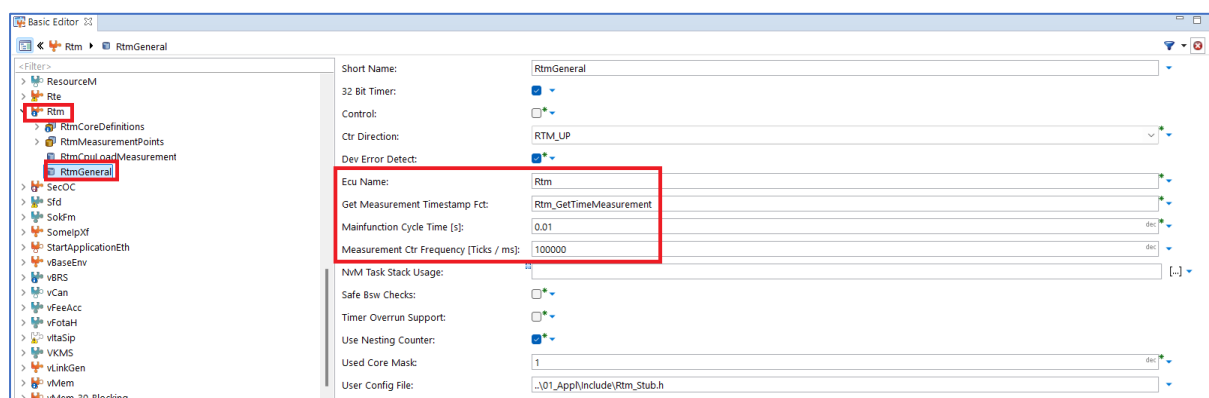


Figure 14

OS Counters Container:

1. The parameter Counter Ticks Per Base [Tick] present in the following path as shown in below image
'MICROSAR/Os/OsCounter' → Counter Ticks Per Base[Tick].

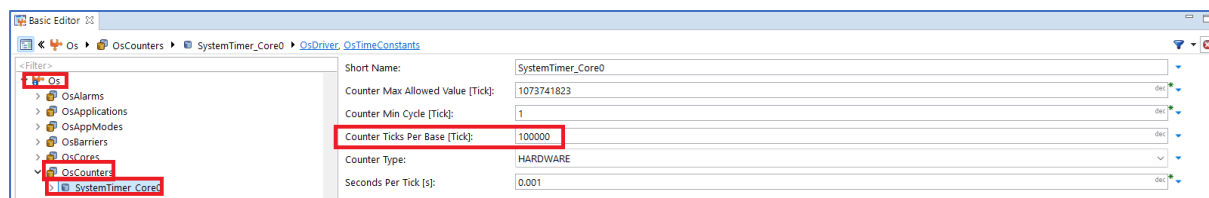


Figure 15

NOTE : The integrator must verify that, 'RtmMeasurementCtrFrequency [Ticks/ms]' should be the same as 'OsCounterTicksPerBase'.

ii. Runtime Measurement Calculation:

1. To measure the runtime of runnables in the system. The recommended way to do this is to configure a MP for each runnable. These MPs can be started and stopped by using VFB trace hook function generated by the RTE.
2. The configuration of MPs, VFB trace functions and their implementation is assisted through the RTE configurator editor, and a script provided by RTM. The necessary configuration steps are explained in the following.
3. Go to VFB Trace, by following the path :
'Configuration Editor/Runtime System General/RTE/VFB Tracing' → Select 'Import VFB Trace Functions Assistant'.

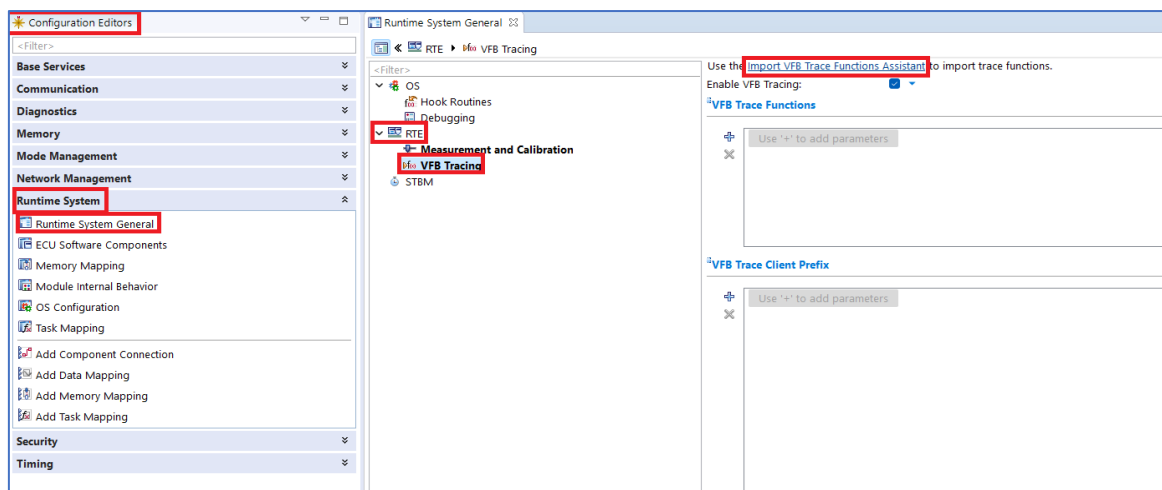


Figure 16

4. In the RTE comfort editor, open the 'Import Assistant' and select the generated file Rte_Hook.h.

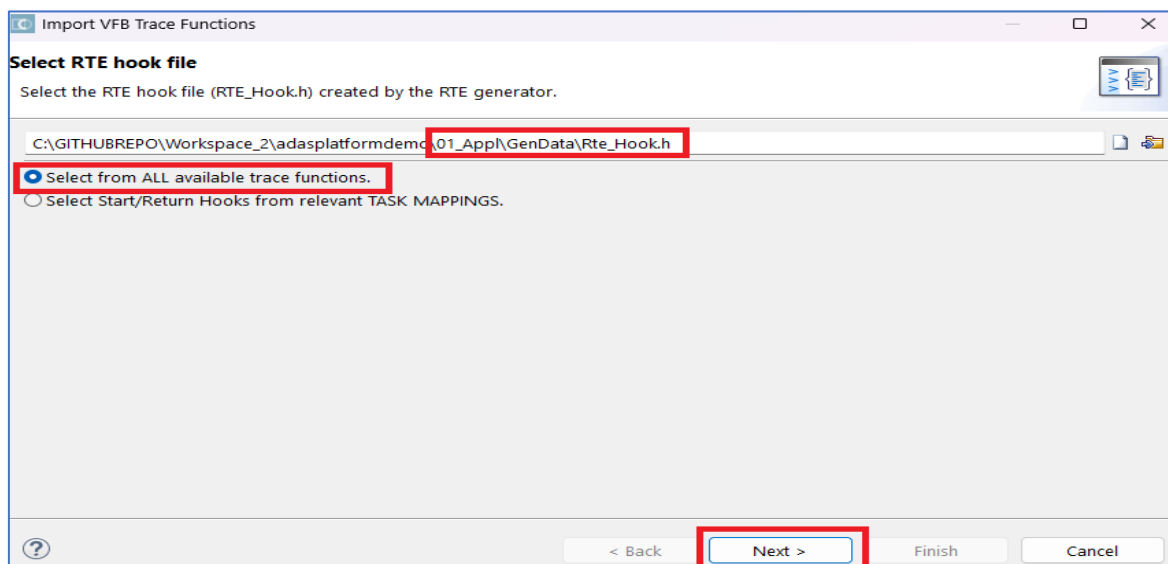


Figure 17

5. Select all Start/Return pairs for those runnable that shall be measured.

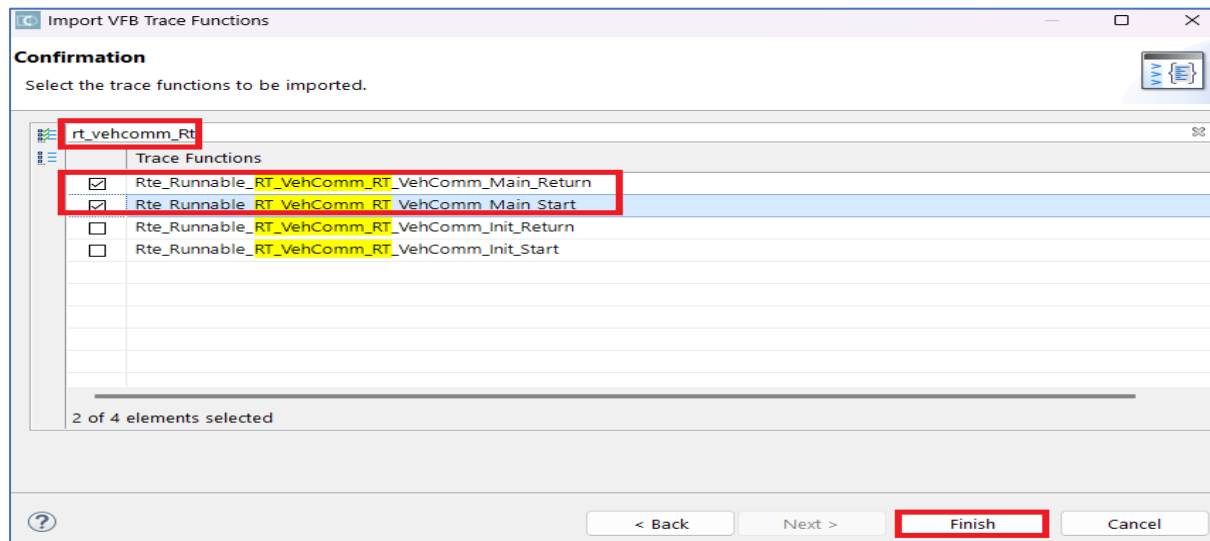


Figure 18

6. Switch to the 'Script Tasks' window, by using the following path: 'Scripting\Script Tasks' as shown in the below image.

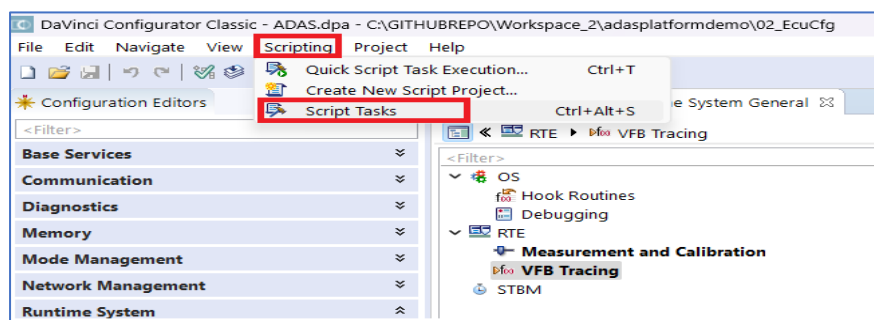


Figure 19

7. Expand the script 'CreateRunnableMeasurementPoints' and right click on the script project. Select 'Execute'.

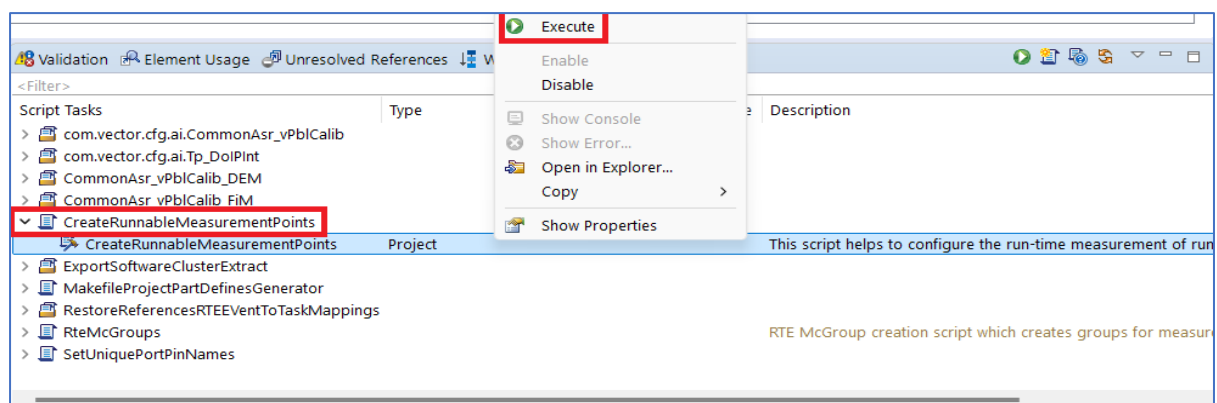


Figure 20

8. The script generates the source file Rte_Hook_Rtm.c in the folder .\Appl\Source. It contains the configured runnable hooks that call the Rtm macros 'Rtm_Start' or 'Rtm_Stop'.

- After Executing the script, Rtm container throws an error message to assign the core for created measurement point and we need to configure some parameters for newly created MP by following the below image.
For Runnables the integrator has to enable the 'Measurement Enabled' parameter and Measurement Type as 'GrossExecutionTime'.

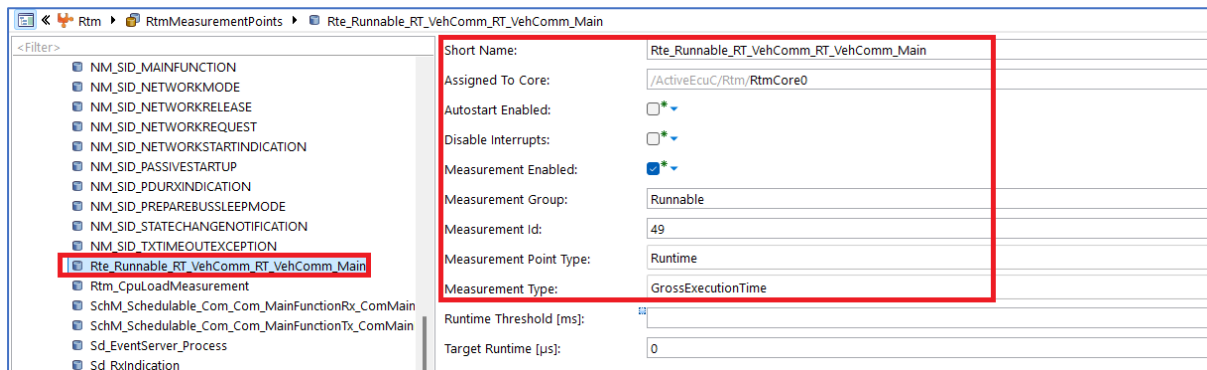


Figure 21

4.1. GetCurrentTime

- For Rtm, GetCurrentTime is one of the main important aspects.
- In our context, we utilized System timer (STM Register).

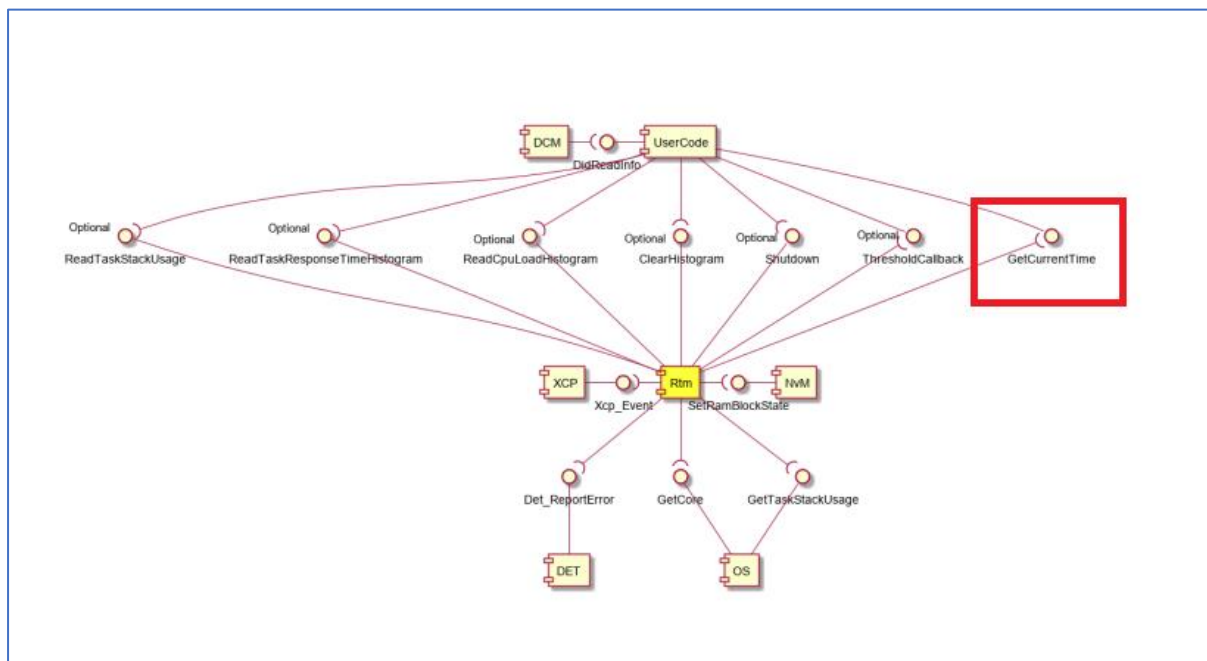


Figure 22

5. Developer Steps

1. The integrator has to create a software component for the timer value, a Software component named **[Appl_Rtm]** has been developed and the Current time is fetched using **Rtm_GetTimeMeasurement** derivation.
2. Follow the below steps for creating software component,
 - i. Open DaVinci Developer -> Home -> Object Browser -> Application Component types -> right click on it and select 'New Application Component Type'.

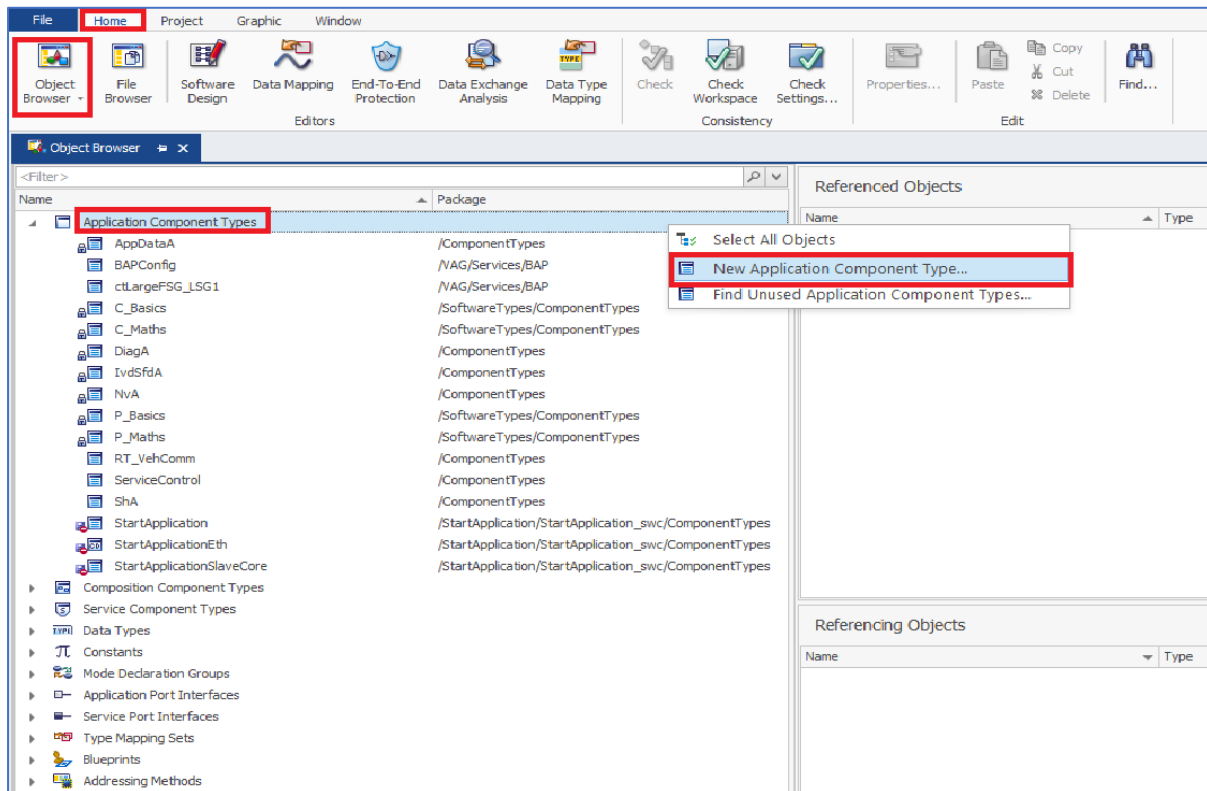


Figure 23

- ii. After creating 'New Application Component type', enter the following values as shown in image below:

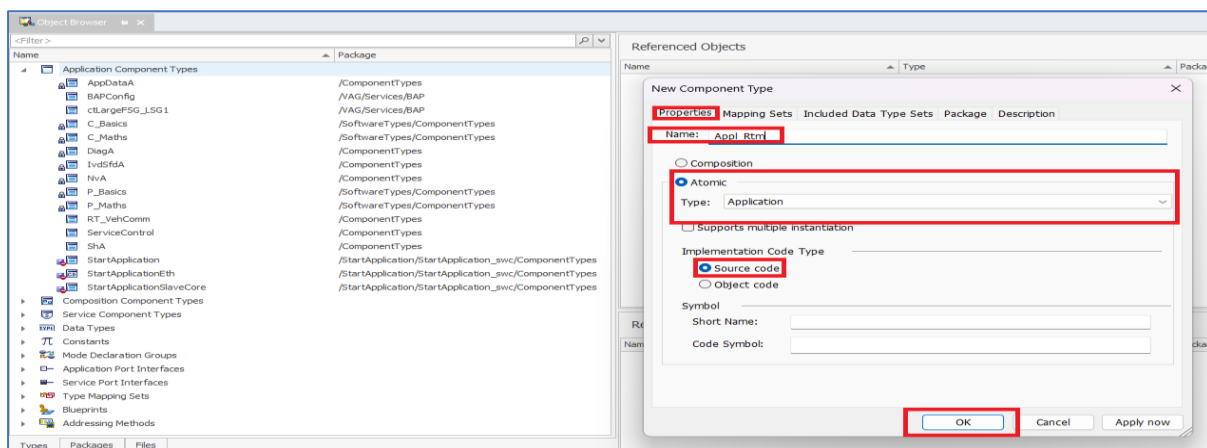


Figure 24

- iii. Once the application component is created 'double click' on it, to navigate to the 'Software Design' In that software design view, click on 'New' located at bottom left corner of the Software Design and select Runnable.

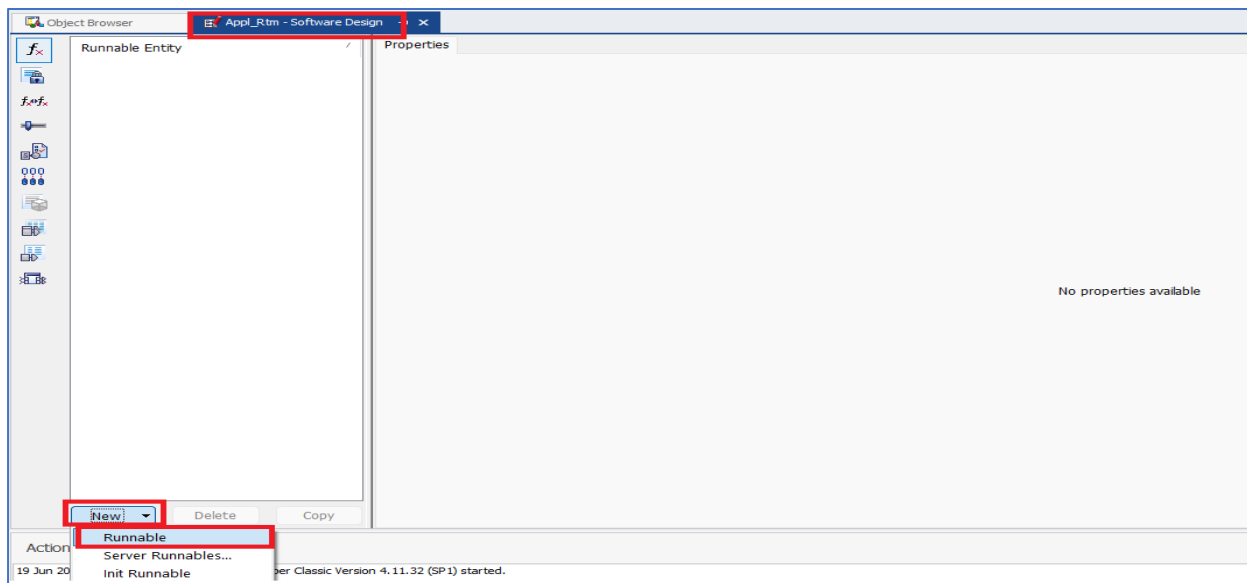


Figure 25

- iv. After creating Runnable, enter the name of the runnable as shown in the image below.

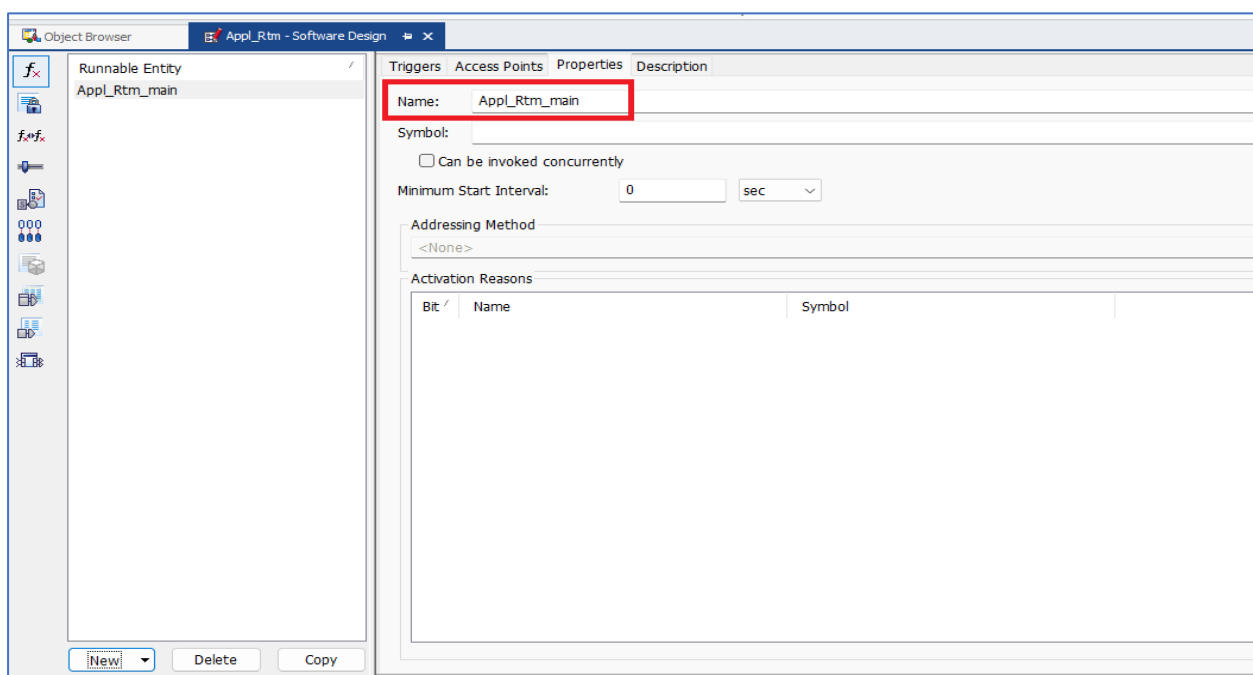


Figure 26

- v. After entering the name of the runnable, the integrator has to give the periodicity for Triggering of the created Runnable.

Follow the below mentioned path for giving periodicity,

Go to Runnable -> Triggers -> click 'New' on right bottom corner -> then select 'Periodic' -> give 'Period' value in msec -> click 'OK'.

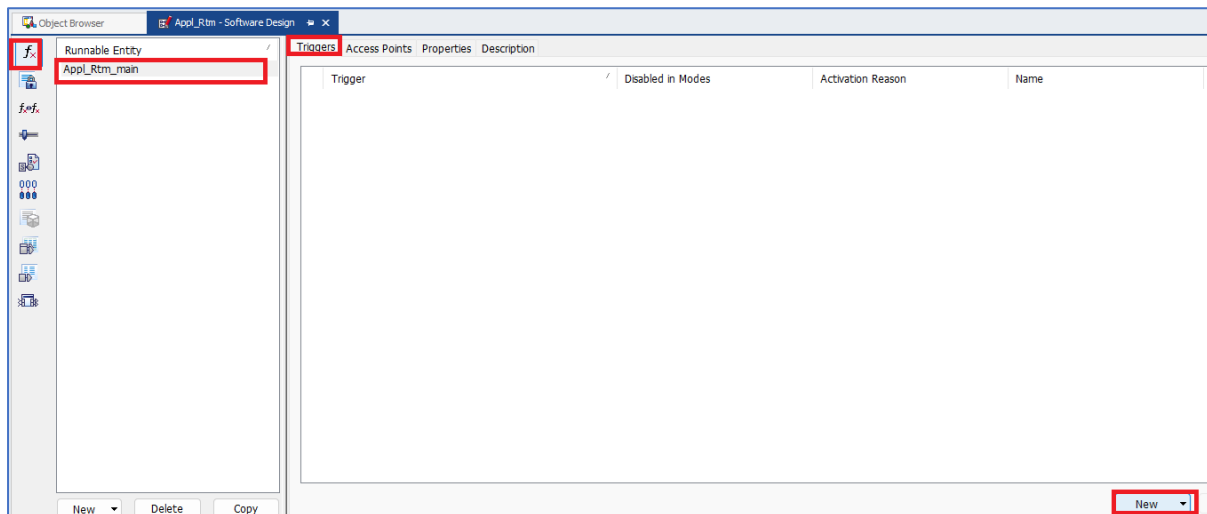


Figure 27

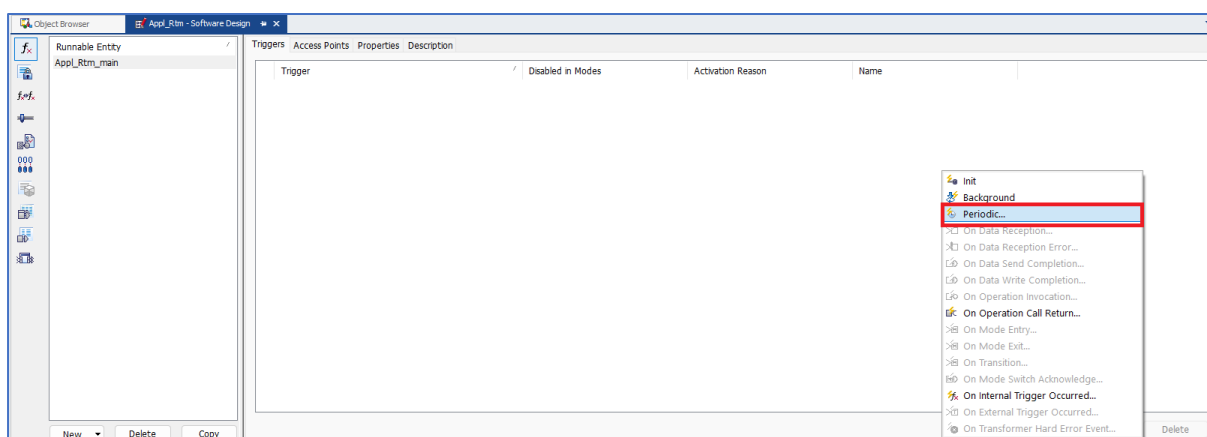


Figure 28

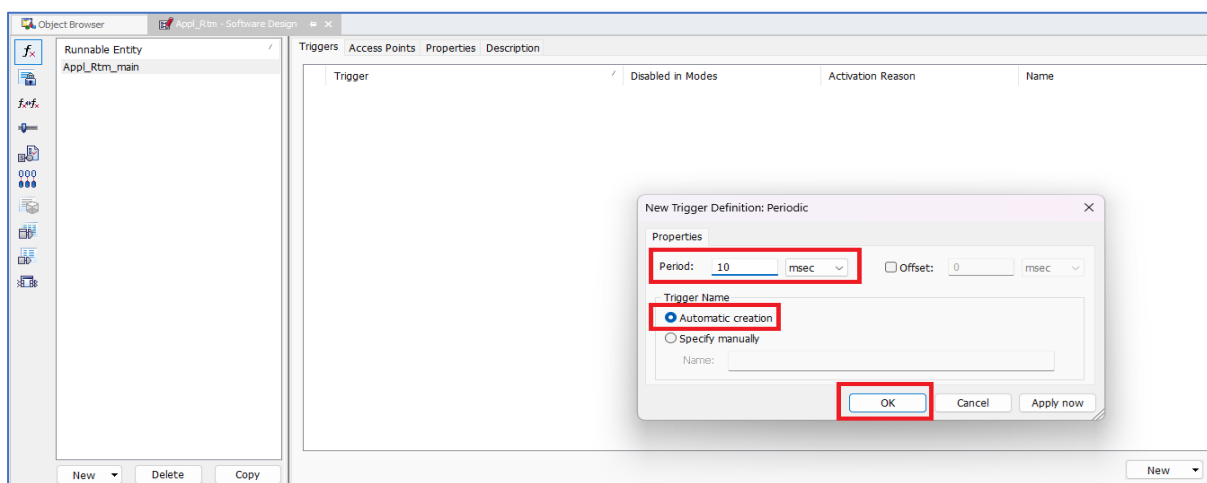


Figure 29

- vi. After creating the application component, open configurator and synchronize the changes and the integrator has to add that application component by following the provided path, Runtime System General/ECU Software Components/Application Components -> Right click on the application component and add created application component.

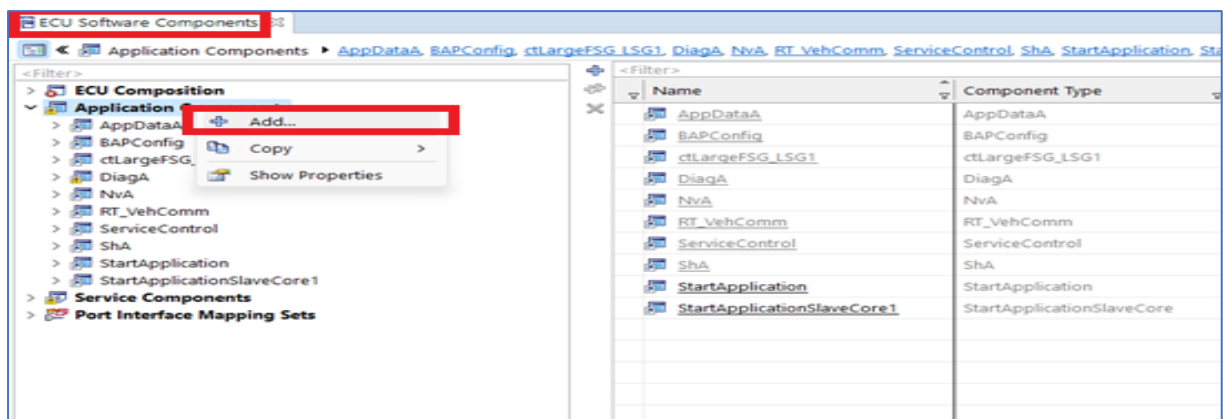


Figure 30

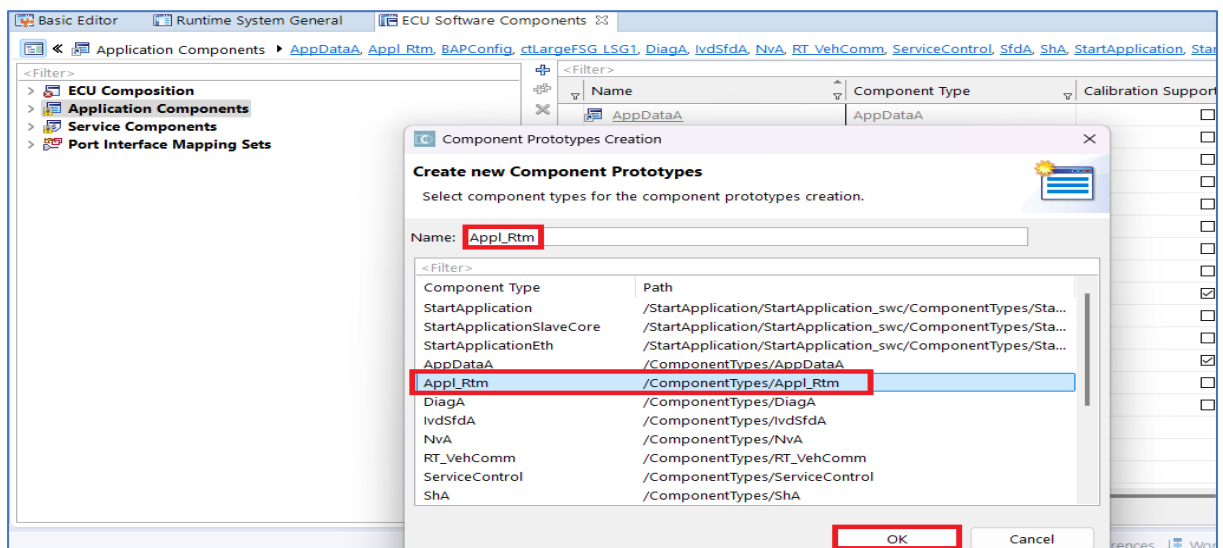


Figure 31

6. Task Mapping & Module Initialization

i. For Task Mapping :

After adding the application component in configuration, the integrator has to perform task mapping for Appl_Rtm_main, Rtm_mainfunction.

For Appl_Rtm_main:

<Filter>				
Position	OsCore0	OsApplication_SystemApplication_Core0	OsApplication_SystemApplication_Core0	O
	Default_Appl_Init_Task Priority 45	Default_Appl_Task Priority 5	Default_BSW_Async_Task Priority 30	
21		RE_Role_Extended	Dem_SatelliteMainFunction	
22		RE_Role_Locked	WdgM_MainFunction	
23		RE_Role_OTA	CanXcp_MainFunction	
24		Appl_Rtm_main	Crypto_30_LibCv_MainFunction	
25		RE_Role_Production	Csm_MainFunction	
26		RE_Role_Superuser	Xcp_MainFunction	
27		BAP_Task	vMemAccM_MainFunction	
28			CanTp_MainFunctionTx	
29			Dbg_PeriodicSamplingFunction	
	Init	Data Reception, Mode Entry, Mode Transition, Periodical/1 ms, Periodical/10 ms, Periodical/50 ms, Periodical/100 ms, Periodical/250 ms, Periodical/1000 ms	Operation Invocation, Periodical/2 ms, Periodical/5 ms, Periodical/10 ms, Periodical/20 ms, Periodical/1000 ms	

Figure 32

For Rtm_mainfunction:

<Filter>				
Position	OsCore0	OsApplication_SystemApplication_Core0	OsApplication_SystemApplication_Core0	O
	Default_Appl_Task Priority 5	Default_BSW_Async_Task Priority 30		
0		Rtm_MainFunction		
1		BswM_MainFunction_OsApplication_SystemApplication_Core0		
2		EcuM_MainFunction		
3		ComM_MainFunction_0		
4	Ctrl	ComM_MainFunction_1		
5	Data	Can_30_Core_MainFunction_BusOff		
6	InternetCommunication	Can_30_Core_MainFunction_Mode		
7	LargeData	Can_30_Core_MainFunction_Read		
8	veCore1	Can_30_Core_MainFunction_Wakeup		

Figure 33

ii. Module Initialization:

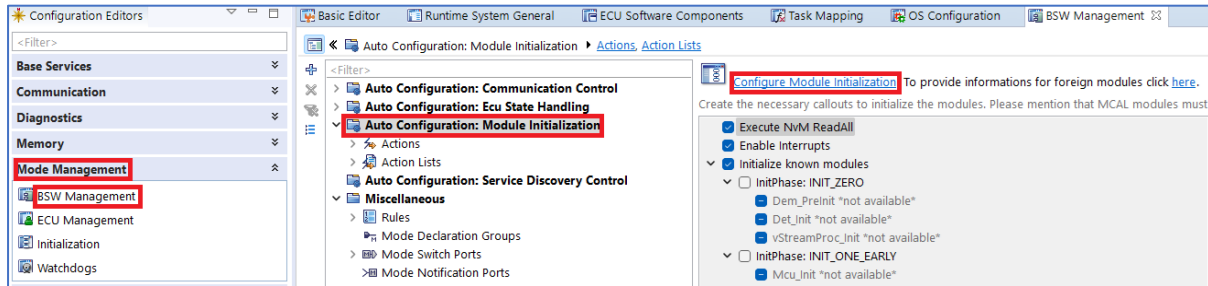


Figure 34

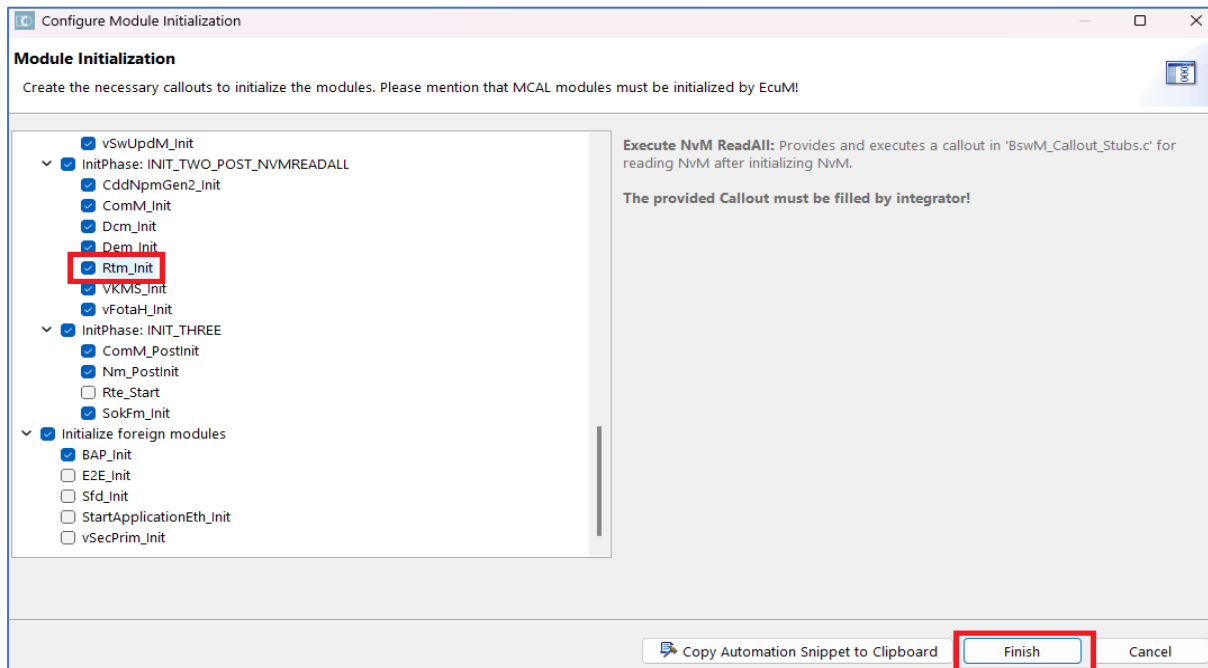


Figure 35

After module initialization, the integrator has to generate 'SWC Templates and Contract Headers' for the created software component. Follow the steps provided below,

- I. In the DaVinci Configurator, click on project tab.
- II. Open the project setting and select the "SWC Templates and Contract Headers" option.
- III. Select the SWC for which we have to generate the template.
- IV. Now select the Generate Software Component Templates and Contract Phase headers option.

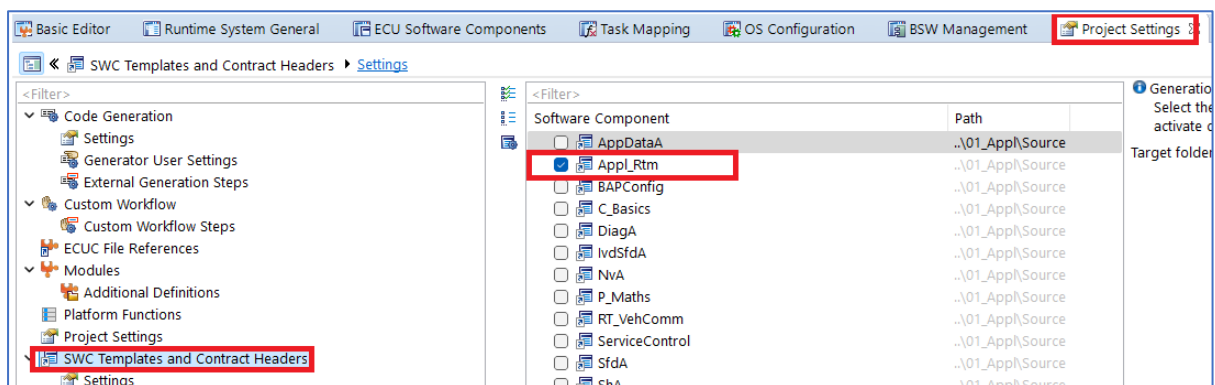


Figure 36

- V. Now click on the Generate Software Component Templates and Contract Headers option and generate the template.

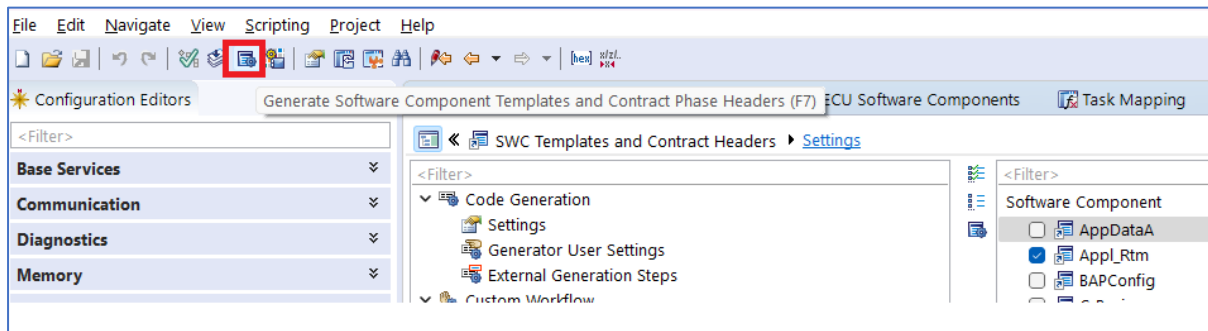


Figure 37

- VI. Once the configuration of RTM module is completed (using all above mentioned steps) next step is to generate the code. In this document we are using DaVinci Configurator for code generation.
- VII. Open the project in configurator.
- VIII. Click on Generate tab as shown below.

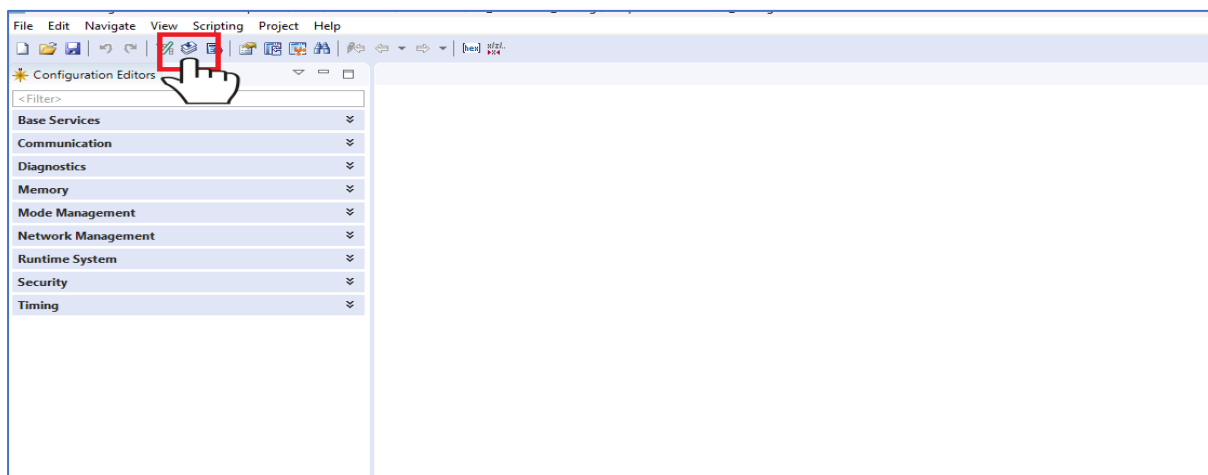


Figure 38

- IX. Select all required modules and generate the code.

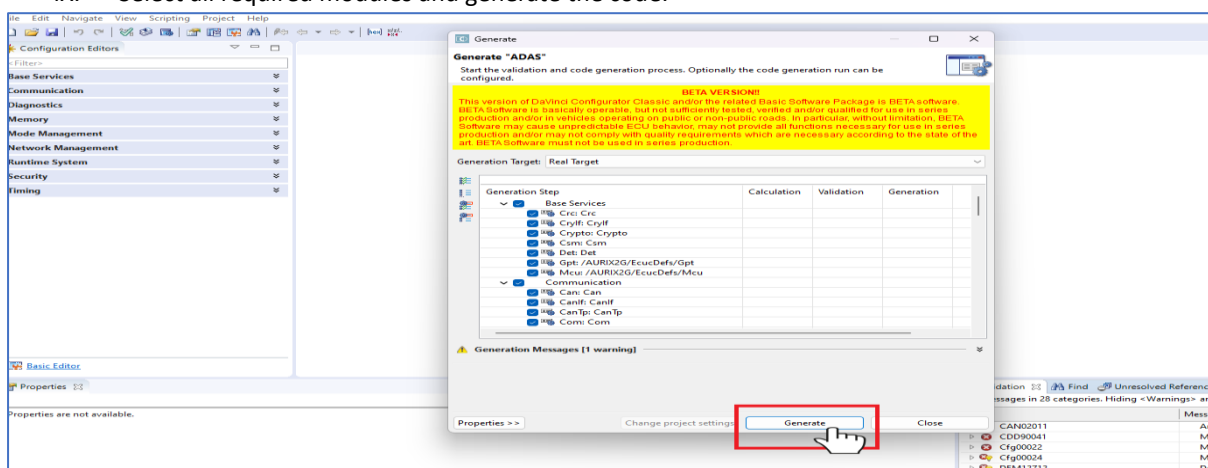


Figure 39

- X. All the generated files are present in GenData directory.

Path:

"{Project_Directory}\01_Appl\GenData\

7. Coding Part

After generating software template, the integrator has to get the Current time and measurement results of the created MP.

1. Include the header files in Appl_Rtm.c file,

```
#include "Rtm.h"
#include "Rtm_Cfg.h"
```

2. To get the measurement results of the created measurements points, include this part of code in Appl_Rtm_main function (Appl_Rtm.c) [Rtm_GetMeasurementItem() -> To get the measurement results]:

```
static uint32 g_CPULoadOverall_u32;
static uint32 g_CPULoadAverage_u32;
static uint32 g_CPUMax_u32;
static uint32 g_CPUMin_u32;
static uint32 g_RTvehcommmain_RunOverall_u32;
static uint32 g_RTvehcommmain_RunAverage_u32;
static uint32 g_ComRxBRunOverall_u32;
static uint32 g_ComRxBRunAverage_u32;
static uint32 g_ComTxRunOverall_u32;
static uint32 g_ComTxRunAverage_u32;
static uint8 g_StartCpuLoad_u8 = 0U;

FUNC(void, Appl_Rtm_CODE) Appl_Rtm_main(void) /* PRQA S 0624, 3206 */ /* MD_Rte_0624, MD_Rte_3206 */
{
    /* DO NOT CHANGE THIS COMMENT! << Start of runnable implementation >> DO NOT CHANGE THIS COMMENT!
    * Symbol: Appl_Rtm_main
    */
    if(g_StartCpuLoad_u8 == 1U)
    {
        Rtm_Start_CpuLoadMeasurement();
        g_StartCpuLoad_u8 = 0U;
    }
    else if(g_StartCpuLoad_u8 == 2U)
    {
        Rtm_Stop_CpuLoadMeasurement();
        g_StartCpuLoad_u8 = 0U;
    }
    else
    {
        ;
    }

    Rtm_GetMeasurementItem(RtmConf_RtmMeasurementPoint_Rtm_CpuLoadMeasurement,RTM_ITEM_CPU_LOAD_CURRENT,&g_CPULoadOverall_u32);
    Rtm_GetMeasurementItem(RtmConf_RtmMeasurementPoint_Rtm_CpuLoadMeasurement,RTM_ITEM_CPU_LOAD_AVERAGE,&g_CPULoadAverage_u32);

    Rtm_GetMeasurementItem(RtmConf_RtmMeasurementPoint_Rtm_CpuLoadMeasurement,RTM_ITEM_MAX,&g_CPUMax_u32);
    Rtm_GetMeasurementItem(RtmConf_RtmMeasurementPoint_Rtm_CpuLoadMeasurement,RTM_ITEM_MIN,&g_CPUMin_u32);

    Rtm_GetMeasurementItem(RtmConf_RtmMeasurementPoint_Rte_Runnable_RT_VehComm_RT_VehComm_Main,RTM_ITEM_RUNTIME_OVERALL,&g_RTvehcommmain_RunOverall_u32);
    Rtm_GetMeasurementItem(RtmConf_RtmMeasurementPoint_Rte_Runnable_RT_VehComm_RT_VehComm_Main,RTM_ITEM_RUNTIME_AVERAGE,&g_RTvehcommmain_RunAverage_u32);

    Rtm_GetMeasurementItem(RtmConf_RtmMeasurementPoint_SchM_Schedulable_Com_Com_MainFunctionRx_ComMainFunctionRx,RTM_ITEM_RUNTIME_OVERALL,&g_ComRxBRunOverall_u32);
    Rtm_GetMeasurementItem(RtmConf_RtmMeasurementPoint_SchM_Schedulable_Com_Com_MainFunctionRx_ComMainFunctionRx,RTM_ITEM_RUNTIME_AVERAGE,&g_ComRxBRunAverage_u32);

    Rtm_GetMeasurementItem(RtmConf_RtmMeasurementPoint_SchM_Schedulable_Com_Com_MainFunctionTx_ComMainFunctionTx,RTM_ITEM_RUNTIME_OVERALL,&g_ComTxRunOverall_u32);
    Rtm_GetMeasurementItem(RtmConf_RtmMeasurementPoint_SchM_Schedulable_Com_Com_MainFunctionTx_ComMainFunctionTx,RTM_ITEM_RUNTIME_AVERAGE,&g_ComTxRunAverage_u32);

    /* DO NOT CHANGE THIS COMMENT! << End of runnable implementation >> DO NOT CHANGE THIS COMMENT!
    */
}
```

Figure 40

In Rtm_Cfg.c file, the integrator able to see the created measurement points in macros and the type of measurement point should be visible in Rtm_Types.h file.

Before starting the measurement point/ Before getting the results, the integrator has to prepare the Measurement points by using Rtm_PrepareMPSettings () API. Follow below code for preparing measurement point settings:

```
Rtm_PrepareMPSettings(RTM_MP_SETTING_ENABLE_ALL,0u); //To Enable Autostart by C_API method//
Rtm_StartMeasurement(0);
```

Figure 40.1

- Please add this part of code to get 'Current Time' using STM timer in Appl_Rtm.c file after Appl_Rtm_main function:

```
static uint32 g_StmTimer_u32 = 0;
static uint32 *g_Rtm_TimerValue_u32;
Rtm_TimestampType Rtm_GetTimeMeasurement(void)
{
    g_Rtm_TimerValue_u32= 0XF0001010; /*Base Address & Offset Address of STM
Timer 0*/
    g_StmTimer_u32 = *g_Rtm_TimerValue_u32;
    return g_StmTimer_u32;
}
```

In the above code, From Rtm_GetTimeMeasurement, we are deriving the timer values from System timer (STM Register) derivation [basically, a direct access via your controller specific free running counter register] to get current time.

STM counter register address = Hardware Timer Base Address STM0 + Offset Address of TIM0 → 0XF0001010.

- Comment this part of code in Demo.c:

```
242  /* ***** */
243  /* Callback function required by Rtm "Runtime Measurement Module" */
244  /* ***** */
245  #define RTM_START_SEC_CODE
246  #include "MemMap.h" /* PRQA S 5087 */ /* MD_MSR_19.1 */
247  /* Using function name RTM_GET_TIME_MEASUREMENT_FCT the configured name will always match */
248  // uint32 RTM_GET_TIME_MEASUREMENT_FCT(void)
249  // {
250  //     uint32 elapsedTime;
251
252  //     elapsedTime = (uint32)Gpt_GetTimeElapsed(GptConf_GptChannelConfiguration_GptChannel_RandomGenerator);
253  // # if (defined( V_PROCESSOR_CANOEEMU ) && !defined( VTTCTRL_CFG_H ))
254  //     /* on CANoeEmu GPT does not work properly => use windows timer instead */
255  //     GetCounterValue(SystemTimer, &elapsedTime);
256  // # endif
257
258  //     return elapsedTime;
259  // }
260  #undef RTM_START_SEC_CODE
```

Figure 41

8. Makefile changes for vector build environment.

In file Makefile.project.part.defines, add the following tags for compilation of RTM module.

File location: '{Project_Directory} \00_Batch\'

File name: Makefile.project.part.defines

Include **Appl_Rtm.c** file:

```
597  
598 #APP_SOURCE_LST += $(SRC_DIR)\BrsHwIntTb.c  
599 APP_SOURCE_LST += $(SRC_DIR)\Appl_Rtm.c  
600 APP_SOURCE_LST += ..\05_MSR\Components\CddNPMGen2\Implementation\CddNpmGen2.c  
601
```

Figure 40

Include **Rte_Hook_Rtm.c** file:

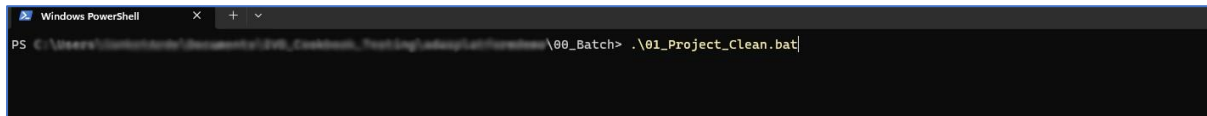
```
690 # add BmHdrHeader, this is necessary so that the bootmanager can find the valid start address for the application.  
691 # in case mapping of BrsStartupEntry is changed, make sure to set it correctly in this structure  
692 #APP_SOURCE_LST += ..\Appl\Source\fb1bmhdr.c  
693 APP_SOURCE_LST += $(SRC_DIR)\Rte_Hook_Rtm.c
```

Figure 41

9. Compilation

After generating code in configurator, next step is to compile the project. For compilation of project follow the below steps.

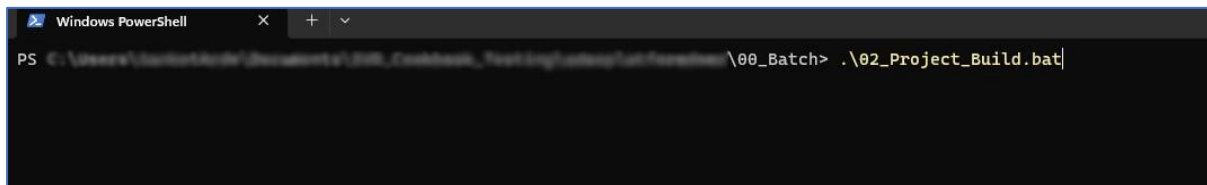
- I. Open directory 00_Batch from project repository.
Path:
“{Project_Directory} \00_Batch\”
- II. Open command prompt (cmd) in present working directory.
- III. Use the below mentioned command to clean the project (This step is not mandatory but is best practice to follow).
Cmd : “01_Project_Clean.bat”



```
Windows PowerShell
PS C:\Users\jankar\Documents\RTM_Cookbook_Testing\adaplatfordev\00_Batch> .\01_Project_Clean.bat
```

Figure 42

- IV. Use below command to build project without cleaning the previous files.
Cmd: “02_Project_Build.bat”

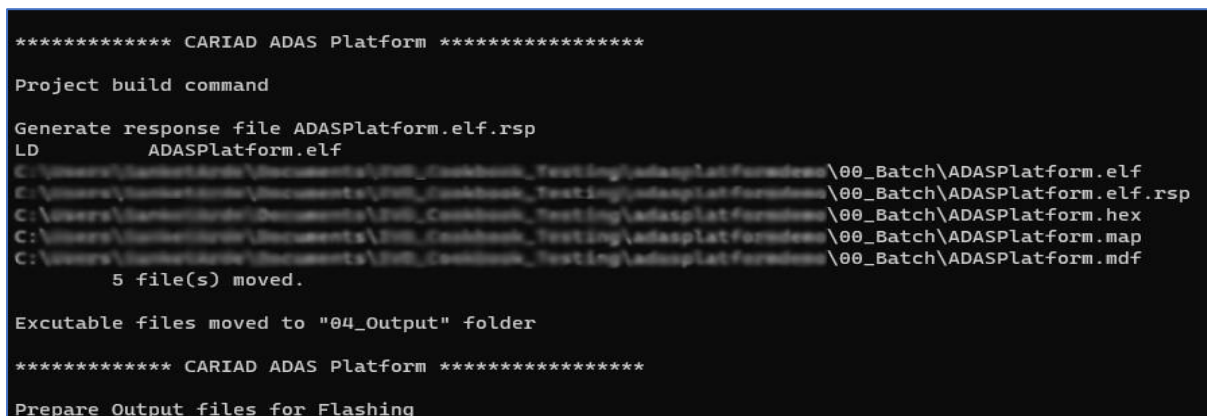


```
Windows PowerShell
PS C:\Users\jankar\Documents\RTM_Cookbook_Testing\adaplatfordev\00_Batch> .\02_Project_Build.bat
```

Figure 43

- V. Once code is successfully compiled, below mentioned files are generated in
“{Project_Directory} \04_Output\” directory.

Files: ADASPlatform.elf
ADASPlatform.elf.rsp
ADASPlatform.hex
ADASPlatform.map
ADASPlatform.mdf



```
***** CARIAD ADAS Platform *****

Project build command

Generate response file ADASPlatform.elf.rsp
LD      ADASPlatform.elf
C:\Users\jankar\Documents\RTM_Cookbook_Testing\adaplatfordev\00_Batch\ADASPlatform.elf
C:\Users\jankar\Documents\RTM_Cookbook_Testing\adaplatfordev\00_Batch\ADASPlatform.elf.rsp
C:\Users\jankar\Documents\RTM_Cookbook_Testing\adaplatfordev\00_Batch\ADASPlatform.hex
C:\Users\jankar\Documents\RTM_Cookbook_Testing\adaplatfordev\00_Batch\ADASPlatform.map
C:\Users\jankar\Documents\RTM_Cookbook_Testing\adaplatfordev\00_Batch\ADASPlatform.mdf
5 file(s) moved.

Executable files moved to "04_Output" folder

***** CARIAD ADAS Platform *****

Prepare Output files for Flashing
```

Figure 44

10. Flashing ECU using TRACE32 debugger

- I. Flashing should be done with the help of .cmm file.
- II. Before proceeding with flashing procedure, update the hex file, elf file, and repository path within the .cmm file as shown below.

Path: "{project_directory} \03_Tools\01_Trace32\"

File Name: tc39x.cmm



Figure 45

- III. Open the TRACE32 debugger.
- IV. In the Target option, Select Tricore and then click on Start.

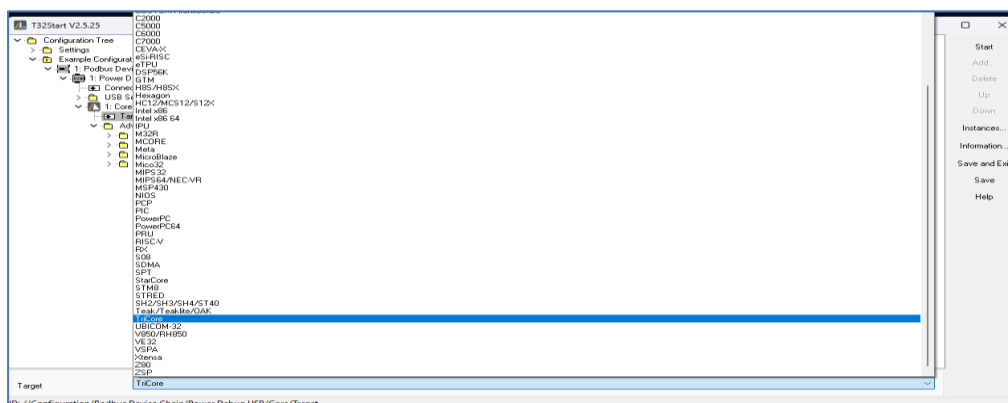


Figure 46

- V. Now, go to file option and select open file to load the updated .cmm file available at location: "{project_directory}\03_Tools\01_Trace32\".

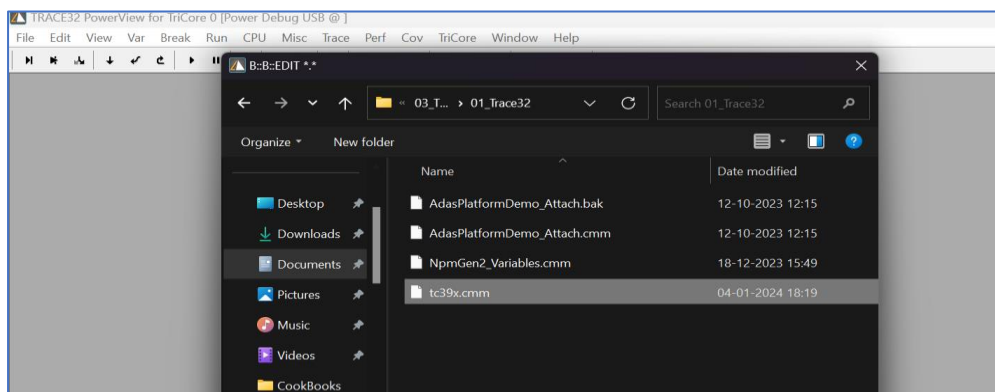


Figure 47

VI. Click the "Do" button. A pop-up window will appear. Click "Yes." ECU flashing will start.

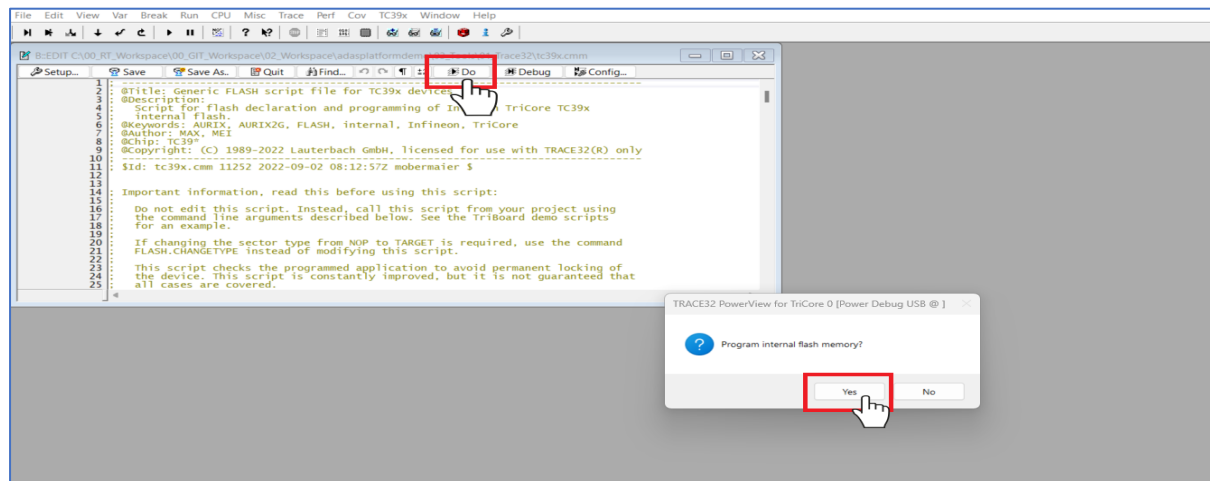


Figure 48

VII. Upon successful completion of the flashing process, "All ok" will be displayed in the window.

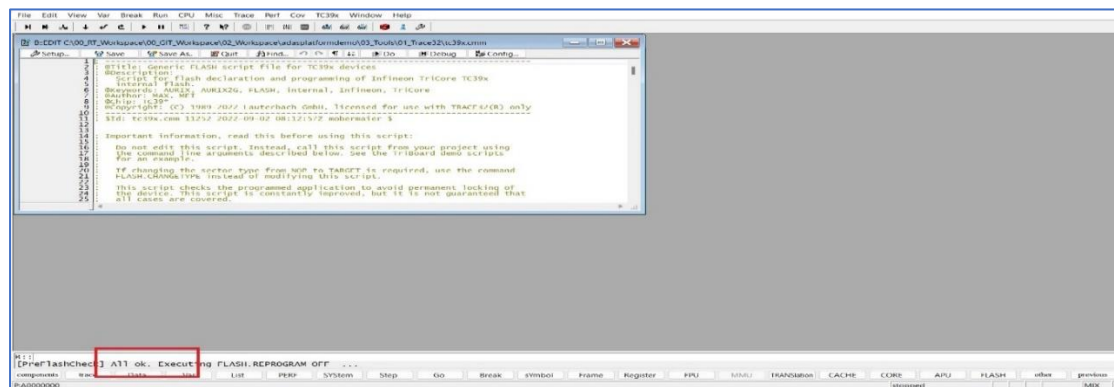


Figure 49

VIII. Click the "Go" button to run the software. User can check the state of software at bottom of window as shown in image.

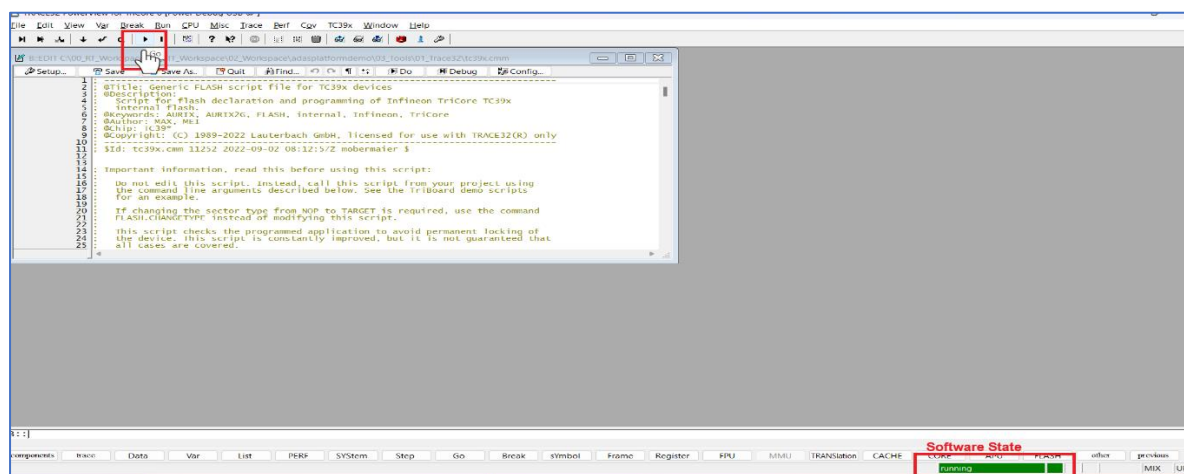


Figure 50

11. Issues observed and their solutions

1. Purpose of the 'Autostart Enabled' Parameter and Alternative Approach

The 'Autostart Enabled' parameter is used to initiate measurements at the start-up of an ECU for specified measurement points. However, enabling this feature can add additional CPU load. Therefore, it is recommended to use this feature for code sections that are executed infrequently.

Alternative Approach Without 'Autostart Enabled':

It is possible to achieve the same result without enabling the 'Autostart Enabled' parameter by using the C_API function `Rtm_PrepareMPSettings`. This function is used for the activation and deactivation of measurement points.

Call ``Rtm_StartMeasurement(C_API)`` immediately after executing the ``Rtm_PrepareMPSettings(MpSetting, ConfiguredMPId)`` function.

2. Why Use STM Timer Derivation Instead of OsCounter for Measurement Results

RTM (Run Time Measurement) requires timestamps from a real-time source, which is obtained directly from the System Timer. Here's why the STM (System Timer Module) timer is preferred:

Advantages of STM Timer:

1. **Direct Access:** The STM timer is a free-running counter register that can be accessed directly using `Rtm_GetTimeMeasurement`, allowing for direct copying of timer values from the System Timer.
2. **Independence from CPU:** Unlike `OsCounter`, which allocates time to tasks and cannot directly access the hardware timer, the STM timer operates independently from the CPU and is duty only for monitoring time.

Using the STM Timer:

1. **Deriving Timer Values:** Use `Rtm_GetTimeMeasurement` to derive timer values directly from the STM register. This provides the current time via direct access to your controller-specific free-running counter register.
2. **Access STM Counter Register:**
 - **Address:** Hardware Timer Base Address STM0 + Offset Address of TIM0
 - So, the address of the STM is '0xF0001010'.

For Offset Address of TIM0, refer 'TC3xx User Manual part2 V2.0 6.pdf'.

By using the STM timer, you can ensure accurate and independent timing for real-time measurement, triggered by callback functions to provide the respective time for that triggered function.

12. Contact